

GROWER M&V Pipeline Guide:

This is a document holding a step-by-step guide on how to run the original code base, meta information about the code, and how to edit the code to your specific needs.

Set-Up:

Folders:

- 'm&v pipeline': holds all of the code
- 'Raw_data': holds all of the GROWER proprietary outage data (accessible upon Data Sharing Agreement)
 - Per_county and per_outage
 - Raw datasets are webscraped from utility websites tracking outages of their customer base
 - Each row is a temporal snapshot of the website where it holds
 - # of people out
 - Current time

Make sure you are using Python 3. This codebase uses Python 3.10.

1. Put the "m&v pipeline" folder in your local directory
2. Navigate into the location of the 'm&v pipeline' folder directory in your terminal of choice.
3. Run 'pip install -r mvp_requirements.txt' to install the 3rd party dependencies

Dependencies:

- Pandas
- Numpy
- PyYAML
- Ruptures

```
pip install -r mvp_requirements.txt
```

4. IF GIVEN ACCESS TO 'raw_data' FOLDER, put said folder somewhere in your local directory. Make sure to keep track of the file path of the 'raw_data' folder
5. Go to [mvp_config.yaml](#) and update the [LOCAL_FILE_BASE_PATH](#) under 'globals' at the top of the file with the path to the raw_data

File Summary:

Important Details to Keep in Mind When Expanding codebase:

- File names follow a very particular format
- Pipeline instance variable names follow particular format
- Must keep the output path as one and one only and if you want to change it, recommended to recreate pipeline instance

`mvp_config.yaml`

File that holds meta utility company information

- 'providers' is a list of structs where each struct holds meta utility company information
 - Name of the utility company, the state the utility company (mostly) operates in, the layout number, the size of the population of customers served, the average customer per county served
- You can edit which utility companies to test given its pipeline subclass is developed by adding or removing a provider struct with the correct information
- Layouts:
 - GROWER organized its data scraping efforts by grouping utility companies operating in the same state with similar website-layouts under the same group
 - Ex: ga1 = every utility company under this group operates in Georgia and are given the arbitrary numeric identifier 1
 - If another state's utility company shares the same layout as another state's group layout (like ga1), that state will get its own identifier (like tx8), BUT the distinct layout groups will be coupled together in internal GROWER data tracking
 - Ex: ga1 and tx8 contain utility companies in GA and TX respectively that share the same website layout. They had the same web-scraping code applied to them. Thus, ga1 and tx8 will be coupled as ga1tx8 where all layouts will follow the same post processing code
 - NOTE: layout numbers are not unique across states
 - Ga4 and TX4 do NOT share the same website-layouts and are in two different layout groups

`base_mvipeline.py`:

General parent pipeline class that holds essential functions and classes used for processing and transforming outage data

- Key Variables:
 - Global:
 - `valid_df_name_set` = {'ground_truth', 'do', 'ganz', 'mixed_threshold', 'changepoints'}
 - Input:
 - `config`: the corresponding struct value holding the utility company meta information

- **base_file_path**: the file path corresponding to where the raw data folder location
 - **start_time**: the date where the filtered dataset will begin
 - Default: "2023-11-01 00:00:00"
 - **end_time**: the date where the filtered dataset will end
 - Default: "2024-11-30 23:59:59"
 - **output_path**: the file path you want to have the [algorithm-aggregated per county dataframes](#), ground-truth dataframe, and comparison table put into
 - The file path is coupled with config
- Instance:
 - **self.config**: holds the **config** struct variable
 - **self.base_file_path**: holds the **base_file_path** variable
 - **self.geomap**: holds the geomap value
 - **self._dataset_lower_bound**: holds the **start_time**
 - **self._dataset_upper_bound**: holds the **end_time**
 - **self._per_outage**: holds the dataframe for the raw per_outage dataset loaded in
 - **self._per_county**: holds the dataframe for the raw per_county dataset loaded in
 - **self.methods_to_run**: holds the list of methods to run where each method is a string that must match one of **valid_df_name_set**
 - **self._ground_truth_df**: holds the dataframe of the groundtruth
 - **self._do_df**: holds the do-algorithm-transformed per_county dataframe
 - **self._ganz_df**: holds the ganz-algorithm-transformed per_county dataframe
 - **self._mixed_threshold_df**: holds the mixed_threshold-algorithm-transformed per_county dataframe
 - **self._changepoints_df**: holds the ChangepointDetection-algorithm-transformed per_county dataframe
 - **self._comparison_table**: holds the comparison table comparing all of the algorithm-transformed per_county dataframes against the ground_truth_df
 - **self._name_to_df**: holds a map between dataframe names and the actual dataframe names for ease of access to the dataframes
 - **self._output_path**: holds the **output_path** variable
- Functions
 - The file defines several functions and classes that are crucial for the pipeline's operation. These include methods for loading, transforming, and aggregating data, as well as applying various algorithms to the data.
 1. Data Loading and Transformation
 - Functions to load data in:
 - **_load_geo_mapping()**

- Loads the local json file to `self.geomap` that maps zipcode to county, county fip, state
- `_load_raw_data()`
 - Loads the raw data outage data (per_outage and per_county) from the 'raw_data' folder for the corresponding utility company
- `_load_aggregated_df(df_name)`
 - Loads the algorithm-aggregated df corresponding to df_name in the local output folder
- `_load_comparison_table()`
 - Loads the locally stored comparison table into `_comparison_table`
- Functions to transform loaded DataFrames based on specific criteria.
 - `_transform_per_outage()`
 - Cleans up and makes consistent the utility company raw per_outage dataset
 - `_transform_per_county()`
 - Cleans up and makes consistent the utility company raw per_county dataset
 - Columns of transformed per_county:
 - 'county'
 - 'customersOutNow'
 - 'customersServed'
 - 'timestamp'
 - 'utilityProvider'
 - `_transform_loaded_aggregated_df(df_name)`
 - Transforms the dataframe loaded from the local csv file and making sure its column attributes are consistent to the algorithm transformed dataset column attribute convention

2. Algorithm Application and Generation

- Functions to apply different algorithms to the data, such as `create_groundtruth()` applied to per_outage, `do_algorithm()` applied to per_county for `do_method()`, `ganz_algorithm` applied to per_county for `ganz_method()`, `mixed_threshold_algorithm()` applied to per_county for `mixed_threshold_method()`, and `changepoints_algorithm()` applied to per_county for `changepoints_method()`
 - These algorithm methods are used to call the respective algorithm in `algorithms.py` for decoupling
- `do_method()`, `ganz_method()`, `mixed_threshold_method()`, and `changepoints_method()`

- Parameters:
 - force_agg=False
 - This is for all algorithm application methods where it allows the user to force the algorithm to be performed regardless whether such a algorithm-transformed dataframe already exists
 - Each method except for changepoints_method() has its own threshold parameter which the user can input
- Every method performs checks whether the corresponding algorithm-transformed dataframe already exists
- If the dataframe does not exist or force_agg = True, the corresponding algorithm is invoked in [algorithms.py](#)
 - The dataframe returned by the algorithm is then stored into the corresponding instance variable
- [create_groundtruth\(\)](#)
 - In basemvpipeline.py, this method simply checks and returns whether a groundtruth dataframe already exists.
 - Since creating the groundtruth differs between utility companies as each company has its own data layout, the logic is implemented in the subclasses in [mvpipeline.py](#)
 - Result Dataset's Column-naming convention for [create_groundtruth\(\)](#)
 - This convention is enforced in the pipeline subclass' create_grountruth
 - Columns
 - 'start_time'
 - 'end_time'
 - 'duration_timestamp_diff'
 - (duration calculation by diff in timestamp)
 - IN NUMERIC REPRESENTING MINUTES
 - 'duration_max'
 - (the duration calculation + 15 minutes that is the greatest)
 - IN NUMERIC REPRESENTING MINUTES
 - 'duration_mean'
 - (average of all duration calculations attempted (excluding duration_approx)

- IN NUMERIC REPRESENTING MINUTES
 - 'duration'
 - (out of all duration calculation attempts, this was deemed the best one)
 - Not named 'duration_approx' for sake of consistency
 - IN NUMERIC REPRESENTING MINUTES
 - 'average_customers_out'
 - 'weighted_customers_out'
 - Riemann sum calculation of customers out * duration to get the total time people were without power
 - IN NUMERIC REPRESENTING MINUTES
 - $\text{avg_customer_out_x_duration} = \text{avg_customers_out} * \text{duration}$
 - 'Utility_provider'
- run_methods(method_list):
 - Runs the methods corresponding to each string value in method_list
- generate_comparison_table(to_compare_list)
 - Generates a comparison table that compares each algorithm-transformed dataframe to each other and the groundtruth
 - To_compare_list = a list of the algorithm aggregated county-level dataframes and/or groundtruth dataframes to add to the table
 - Assumes that either the algorithm-applied dataframes and/or groundtruth is already generated and stored in the instance variables or is stored locally in the 'output' directory
 - A row of summary statistics is made for each dataframe:
 - Type of the model (aka the algorithm)
 - Name of the model (aka specific threshold values passed in)
 - Frequency (the number of outages calculated)
 - Average duration (the average duration across all outages)
 - Average customers out (the average of the average customers out)

- Total customer outage time (the sum of the weighted customers out across all outages)
- Average customer outage time (total customer outage time / frequency)
- Frequency__diff (the delta between the algorithm aggregated county-level data's frequency and the groundtruth's frequency for the utility company)
- Average_duration_diff ((the delta between the algorithm-applied dataframe's average duration and the groundtruth's average duration for the utility company)
- Total_customer_outage_time__diff (the delta between the algorithm aggregated county-level dataframe's Total customer outage time and the groundtruth's Total customer outage time for the utility company)
- Frequency__percentage_difference (the percentage difference of the frequency between the algorithm-applied dataframe and the groundtruth)
- Average_duration__percentage_difference (the percentage difference of the average duration between the algorithm aggregated county-level dataframe and the groundtruth)
- Total_customer_outage_time__percentage_difference (the percentage difference of the total_customer_outage_time between the algorithm-applied dataframe and the groundtruth)
- For a more essay-like explanation of the comparison tables, refer to the high-level explanation [doc](#).

3. Utility Functions

- Helper functions for file operations, date manipulations, and other common tasks.
- Constructors and Setters
 - `_construct_raw_file_path()`
 - Constructs the global file path of the 'raw_data' folder for the specific utility company indicated in 'config'
 - `_set_output_path(output_path)`
 - Sets the instance `output_path` variable
 - Must keep the output path as one and one only and if you want to change it, recommended to recreate pipeline instance
 - `_set_timeframe()`

- Sets the beginning (dataset_lower_bound) and end (dataset_upper_bound) of the data to be processed for the raw data
- `_construct_exported_df_path(df_name)`
 - It takes in a specific dataframe name (ground_truth, do, ganz, mixed_threshold, changepoints, etc.) (which has to match one of 'valid_df_name') and generates a path for the specified dataframe (to export to or to retrieve from)
- `_construct_comparison_table_path()`
 - Constructs the path for the comparison table generated
 - Path should have the same directory as the other algorithm-transformed dataframes and groundtruth
- `_construct_and_get_df_variable(df_name)`
 - Pass in the dataframe name and the dataframe stored in the corresponding instance variable is returned
- `set_methods_to_run(list):`
 - sets self.methods_to_run = list
- Exporting Files:
 - `export_file(df_name:valid_df_name, output_folder = None, replace = False)`
 - df_name = the string associated with the dataframe stored in the corresponding instance variable you want to export
 - Output_folder = the folder you want to output the file to
 - Should be under the parent folder 'output'
 - replace = Whether to replace the file(s) already in output folder with the dataframe you want to export
 - `export_comparison_table(output_folder, replace)`
 - Output_folder = the folder you want to output the file to
 - replace = Whether to replace the file(s) already in output folder with the comparison table you want to export
 - This method exports the comparison table stored in the **self._comparison_table** to the output folder

`mvpipeline.py`:

Child pipeline class that holds custom subclasses of `base_mvpipeline.py` where each subclass is associated with a unique layout. overrides `base_mvpipeline.py` variables and methods for the specific layout.

- Implements custom transformation methods for different data processing needs for each layout-subclass coupling
- As of 12/2024, 3 layouts included:
 - GA1TX8 Class
 - GA11TX12 Class
 - GA3TX16 Class
- Each subclass-layout coupling differs in its transformation of the raw per_outage data (`_transform_per_outage`), transformation of the raw per_county data (`_transform_per_county`), and the `create_groundtruth()` method

`algorithms.py`

File holding the algorithms that are applied to the raw [per_county](#) file to turn it into [algorithm aggregated county-level data](#)

- Each method in the file corresponds to an algorithm and its logic
- Each algorithm is explained at a high level in this [document](#) or the [methods](#) section.
- Each method is invoked by the corresponding method in `base_mvpipeline.py`
- Each method (except for ChangePoints) that has a threshold (Do's, Ganz, Mixed_Threshold) can have different threshold values passed in.
- Result dataset's column naming convention for

```
do_algorithm(), ganz_algorithm(),  
mixed_threshold_algorithm(), and  
changepoints_algorithm()
```

- **(No indexes)**
- 'county'
- 'start_time'
 - Calculated start time of the outage
 - In Timestamp
- 'end_time'
 - Calculated end time of the outage
 - In Timestamp
- 'duration'
 - Calculated duration of the outage
 - IN NUMERIC REPRESENTING MINUTES
- 'average_customers_out'
 - Average customers out calculated during the outage
 - IN NUMERIC REPRESENTING MINUTES
- 'weighted_customers_out'
 - Total life time spent without power

- Average_customers_out * duration
 - IN NUMERIC REPRESENTING MINUTES
 - 'customers_served'
 - Customers served
 - 'utility_provider'
- Each algorithm is tailored to the code base and is NOT completely modular for external use. To learn how to use the algorithms for your datasets, reference [Running the Algorithms on Your Own Data](#).
- Expandable with the option to add more algorithms
 - When adding new algorithms:
 - Determine name: <insert_name> (MUST KEEP THIS THE SAME)
 - Add as: "def <insert_name>_algorithm to algorithms.py
 - Must also add corresponding method invoking to the Pipeline:
 - Follow same naming convention as others:
 - "def <insert name>_method():
 - Update valid_df_name Literal and valid_df_name_set with <insert name>
 - <insert name> has to be consistently the same everywhere
- For `changepoints_algorithm()`, logic of the ChangePointDetection is not implemented here but instead in a separate `ChangePointDetection.py` file where its dataset generating algorithm is invoked by `changepoints_algorithm()`
 - ChangePointDetection was too large to fit in a single method, so I dedicated an entire file to that method

ChangePointDetection.py

File defining a class for detecting change points in outage data. To get a high level explanation of how the algorithm logic works, refer to the [method list](#).

- An instance of ChangePointDetection takes in two training datasets
 - `Train_aggregated_per_outage`
 - The groundtruth dataset
 - `Train_per_county`
 - The raw per_county dataset to apply the ChangePointDetection algorithm to
- You run `run()` to go through all of the models and find the best penalty values for each model. For each model and unique penalty value, the instance applies the ChangePointDetection method with the specific model and penalty value to the per_county dataset and determines based on how close the frequency approximates the training dataset (AKA train_aggregated_per_outage) which penalty values are the best.

- Instance Variables:
 - Best_frequency_diff
 - Type: float
 - Description: Stores the best (smallest) frequency difference found during the change point detection process. Initialized to positive infinity (float('inf')) to ensure any actual calculated difference will be smaller.
 - Best_fre_mod
 - Type: str
 - Description: Stores the best model used for frequency change point detection. Initialized to 'l2', which is a common model used in change point detection algorithms.
 - Best_fre_pen
 - Type: float
 - Description: Stores the best penalty value used for frequency change point detection. Initialized to 0.1.
 - Best_fre_change_points
 - Type: dict
 - Description: Stores the best change points detected for frequency changes. Initialized to an empty dictionary.
 - Mean_durations_seconds
 - Type: float
 - Description: Stores the mean duration of events in seconds. Initialized to positive infinity (float('inf')).
 - Answer_avg_duration
 - Type: float
 - Description: Stores the average duration of events. Initialized to positive infinity (float('inf')).
 - Answer_frequency
 - Type: float
 - Description: Stores the frequency of events of the training dataset. Initialized to positive infinity (float('inf')).
 - Answer_avg_customer_affected_mean
 - Type: float
 - Description: Stores the average number of customers affected by events of the training dataset. Initialized to positive infinity (float('inf')).
 - Answer_total_customer_affected_mean_duration
 - Type: float
 - Description: Stores the total mean duration of events affecting customers of the training dataset. Initialized to positive infinity (float('inf')).
 - Total_customer_affected_x_duration
 - Type: float

- Description: Stores the product of the total number of customers affected and the duration of events. Initialized to positive infinity (float('inf')).
 - Results_by_county
 - Type: None (initially)
 - Description: Stores the results of the change point detection process by county. Initialized to None.
 - Change_points_by_county
 - Type: None (initially)
 - Description: Stores the change points detected for each county. Initialized to None.
- Constructor Parameters
 - Aggregated_per_outage
 - Type: DataFrame
 - Description: Aggregated data per outage for the utility company aka the groundtruth dataframe.
 - Per_county
 - Type: DataFrame
 - Description: the raw per_county data for the utility company
 - Start_date
 - Type: Timestamp
 - Description: The start date for the analysis, localized to US Eastern Time. Default is January 1, 2023.
 - End_date
 - Type: Timestamp
 - Description: The end date for the analysis, localized to US Eastern Time. Default is December 31, 2023.
- Methods:
 - detect_outages()
 - Parameters
 - Data: the data to test the run the model on (per_county data)
 - model: The change point detection model being applied to find the changepoints.
 - penalty: A penalty value used in the change point detection algorithm to control the number of detected change points.
 - Purpose: Evaluates the performance of a specific change point detection model by calculating changepoints and thus outages to determine how well the model identifies change points in the data
 - Functionality: Uses the ruptures library to calculate the changepoints and thus the outages
 - Returns a list of changepoints and then an aggregated dataframe with outage instances as the ChangePointDetection-algorithm-aggregated per_county data

- `evaluate_model()`
 - Parameters
 - Data: the data to test the run the model on (per_county data)
 - Mod: The change point detection model to be evaluated.
 - Pen_val: A penalty value used in the change point detection algorithm to control the number of detected change points.
 - Functionality: Modularized function that invokes `detect_outages` and returns `detect_outages`' return values
- `binary_search_best_penalty()`
 - Parameters:
 - Data: The train_per_county dataset to find the best penalty values for
 - Mod: the model to be applied to the train_per_county
 - Pen_min: the minimum penalty value to prevent infinite searching
 - Pen_max: the maximum penalty value to prevent infinite searching
 - Tolerance: the value that has to be less than pen_max - pen_min
 - Functionality: Function that performs a binary search of penalty values invoking `evaluate_model()` to find the best penalty value for the given mod
- `run()`
 - Parameters:
 - Models: ["l2", "rbf", "linear", "l1", "normal", "rank", "ar"] the different models to find changepoints with,
 - Pen_min: the minimum penalty value to prevent infinite searching
 - Pen_max: the maximum penalty value to prevent infinite searching
 - Tolerance: the value that has to be less than pen_max - pen_min
 - Functionality: Goes through each model in models and performs `binary_search_best_penalty()` to find the best penalty values for each model
- `Filter_data()`
 - Parameters:
 - Aggregated_per_outage: the groundtruth dataset
 - Per_county: the per_county dataset
 - Start_date: the date to mark the start of the filtered data
 - End_date: the date to mark the end of the filtered data
 - Functionality: Filters out the data for both aggregated_per_outage and per_county that is within the start_date and end_date

- `evaluate_best_model()`
 - Parameters:
 - Aggregated_per_outage: the groundtruth dataset
 - Per_county: the per_county dataset
 - Start_date: the date to mark the start of the filtered data
 - End_date: the date to mark the end of the filtered data
 - Functionality: Filters the aggregated_per_outage data and applies the stored best model and penalty value to it
- `demonstrate_best_model()`
 - Parameters:
 - None
 - Functionality: Applies the best change point detection model to the data and segments it based on the detected change points. It iterates over each county and city combination, labels segments of data between change points as distinct outages, and assigns unique outage identifiers. If no change points are detected for a county and city, the entire data for that combination is labeled as a single outage. The method returns the segmented data with the new labels.
- **WARNING:** Only `evaluate_model()` and `detect_outages()` were adjusted and used for the pipeline MEANING the other methods would not work with the pipeline. To figure out how to adjust all methods to your liking and your own datasets, refer to [Running the Algorithms on Your Own Data](#)

`generate_dataframes.py()`

- It goes through each utility company listed in `mvp_config.yaml`, creates a pipeline for each, and generates + exports algorithm-aggregated dataframes for each
- While you can edit what threshold parameters to pass in for each method each time you run a method, you CANNOT run the same method multiple times with different threshold values within the same instance of the pipeline.
- You can run each method with a unique threshold value once each, but if it is NOT the default threshold values, the resulting comparison and model rankings tables will not reflect that accurately
- WARNING: To generate all dataframes for all utility companies listed in `mvp_config.yaml`, it takes around 12 hours with a single-core processor. Using an HPC is highly recommended.

`generate_comparison_table.py()`

- It goes through each utility company listed in `mvp_config.yaml`, creates a pipeline for each, and runs `generate_comparison_table(to_compare_list)` for all of the algorithm-applied dataframes for each

- Reminder: You need to have the algorithm-aggregated dataframes and/or groundtruth already generated and stored in the instance variables or locally in the 'output' folder

generate_model_rankings.py()

- It goes through each utility company listed in `mvp_config.yaml`, concatenates all of the comparison tables into a `merged_comparison_table` (exported)
- It groups the `merged_comparison_table` by algorithm type (groundtruth, do, ganz, mixed_threshold, changepoints) and by algorithm type specific models (unique threshold for do, ganz, mixed_threshold and the different segmentation models used in ChangePointDetection)
- For each group, it takes an average of all of the diff values, and a rank is given to each average diff category based on how low the average diff is. With 6 average diffs, 6 rankings for each group is given.
- A final model ranking is calculated by taking a weighted sum of the 6 rankings, and the each group is given a final sum ranking.
- The lower the sum ranking, the closer that corresponding model type + model name approximates the groundtruth

Running Original Pipeline:

WARNING: If you do not have access to the 'raw_data', you CANNOT run the original pipeline unless you have direct replacements

The pipeline takes in a datasets tracking outages of customers under a certain utility company, calculates outage frequency, duration, # of people affected, etc. with various methods and allows comparison to determine which method is the most accurate

Replicating Test Model Comparison Table Shown Below

1. Take a look at `mvp_config.yaml`.
 - a. The way `mvp_config.yaml` is originally will have all tested utility companies' data be processed.
 - b. If you want to control which utility companies you test for, simply comment out the structs holding the utility companies whose data you don't want to process
2. If you are not given the exported dataframes, run `generate_dataframes.py()` to load the raw data, generate, and export the dataframe to your local directory. IF YOU ALREADY HAVE THE EXPORTED DATAFRAMES, GO TO STEP 3.
 - a. If you are rerunning the file or have previously generated dataframes already exported, you can toggle the 'force_agg' flag for each algorithm-invoking method or the 'replace' flag for the `export_fil()` function to prevent the algorithms being run again or prevent the files from being exported

- b. WARNING: Expected runtime for all utility companies listed in `mvp_config.yaml` is up to 12 hours with a single-core processor. Using an HPC is highly recommended.
3. After generating and exporting the dataframes, run `generate_comparison_table.py` to generate the comparison table for all of the utility companies listed in `mvp_config.yaml`
 - a. If you edit `mvp_config.yaml` after running `generate_dataframes.py()`, make sure you rerun `generate_dataframes.py` before running `generate_comparison_table.py`
4. After generating the comparison tables, run `generate_model_rankings.py()`
5. If you go to your 'output'/'transformed_data' folder within the m&v pipeline folder, you should see a table ranking which algorithm best approximated the groundtruth

Running Additional Tests on Your Own:

- To run tests on your own, you can edit `mvp_handler.py` and the `test()` function to test different values.
 - You can copy over code from `generate_dataframes.py` and `generate_comparison_table.py` and `generate_model_rankings.py`
 - To tweak the threshold values for each method (except for Changepoints), you can simply pass in a different threshold value when you run the method
- (12/2024) Current codebase does not support running the same method multiple times with tweaked threshold values
 - If you want to test multiple different threshold values, you will have to regenerate the comparison tables and model rankings every time
- (12/2024) While the current code base does support running one instance of each method per pipeline, the resulting comparison table and model rankings will not reflect the correct threshold values IF you use any value besides the default thresholds.
- Feel free update the codebase to resolve these bugs and handle the performance you require.

Running the Algorithms on Your Own Data:

The most important part of the codebase are the algorithms applied to county-level data. Here is some pointers of how to modularize `algorithms.py` for your own dataset

- Prerequisites:
 - The outage data you want to test the algorithms on needs to be **county-level** data where the data gives county information of where it outage data comes from
 - Each row of the data must be a **time snapshot row** or something similar
 - Each row is NOT an completed outage event. Each row is PART of an outage event
 - Ex: Raw `per_county` dataset before `transform_per_county()` applied

	name	customersAffected	customersOutNow	customersServed	timestamp	EMC
0	Taylor	2	2	2536	06-28-2023 20:40:26	Flint EMC
1	Houston	5	5	68913	06-28-2023 20:40:26	Flint EMC
2	Taylor	2	2	2536	06-28-2023 20:55:22	Flint EMC
3	Houston	5	5	68914	06-28-2023 20:55:22	Flint EMC
4	Taylor	2	2	2536	06-28-2023 21:10:23	Flint EMC

- Columns
 - Certain columns need to be present and with a specific naming convention:
 - 'county'
 - The county the time snapshot row represents
 - 'timestamp'
 - The time the row's data was reported
 - 'customersOutNow'
 - The reported customers without power at 'timestamp'
 - 'customersServed'
 - The reported customers that are serviced by the utilityProvider
 - This attribute is not absolutely needed, but you will need to edit out the code with this attribute if you do not include customersServed
 - 'utilityProvider'
 - The name of the utility company
 - These columns are consistent across all algorithms except `changepoint_algorithm()`. As long as you rename the columns that best correspond to the 5 above to those exact column names, the code will not produce Key Errors
 -
- Adjusting the algorithm methods:
 - For `do_algorithm()`, `ganz_algorithm()`, and `mixed_threshold_algorithm()`, make sure rename the columns of your dataset to match the columns listed above.
 - For `changepoint_algorithm()`, refer to 'Adjusting ChangePointDetection.py'
 - Each method takes in an instance where each instance is a pipeline.
 - To decouple the method from the pipeline codebase, simply remove the instance parameter and REPLACE `instance._per_county` with whatever county-level dataset you want to apply the algorithm to
 - If you want to use `changepoints_algorithm()`, make sure to have a groundtruth dataset to replace `instance._ground_truth_df` and adjust the columns accordingly
- Adjusting [ChangePointDetection.py](#)
 - If you want to use the `ChangePointDetection` class for your own dataset:
 - Make sure you have a county-level dataset to treat as the training set

- Required Columns:
 - 'CustomersOut' = number of customers without power at that 'RecordDateTime'
 - 'CustomersTracked' = number of customers serviced by the utility company
 - 'RecordDateTime' = time the row's data was reported
- Make sure you have a groundtruth dataset to find the best model and penalty values for
 - Required columns:
 - 'start_time' = start time of the outage
 - 'end_time' = end time of the outage
 - 'duration' = duration of each outage
 - 'customer_affected_mean' = the (average) number of customers affected
 - 'cust_affected_x_duration' = the product of 'customer_affected_mean' and 'duration'
 -
- This codebase uses precalculated penalty values for each model and applies them to all datasets (see 'best_model_hist' in [change_points_algorithm\(\)](#) in [algorithms.py](#))
 - To generate your own penalty values that may be a better fit for your dataset than the predetermined ones, initialize a `ChangePointDetection` instance with the two training sets (`aggregated_per_outage` and `per_county`) and invoke [run\(\)](#)
- After invoking [run\(\)](#), invoke [evaluate_model\(\)](#) with the best model and penalty value to output the aggregated-per-county dataset

Glossary:

- **Algorithms:** the various [methodologies](#) GROWER researched that tries to estimate outage instances from per_county data
 - Do, ganz, mixed_threshold, changepoints
- **Algorithm aggregated county-level data:** taking raw county-level data and applying a method to convert it to outage-level data
- **per_county:** outage data where it reports outage data at the county-level visibility. Only data about the number of customers out in the county at a certain time is available.
 - GROWER's county-level data is **temporally ordered** where every row of the dataset is a timestamp with the number of people without power in what county
 - Also referred as *per_county* data
- **Ground truth:** The aggregated outage-level data declared as the true aggregated outage-level data of a dataset and used as a basis of comparison to see which methods best approximate said ground truth
- **per_outage:** raw temporally-ordered outage data where every row of the dataset is a timestamp with outage identification and the number of people without power in what locations
- **Time snapshot row:** A row in the (raw) outage-level and (raw) county-level data where it the data representation of the web-scraper data collection attempt of the utility provider's website at the recorded time
 - For any raw data used, such a row contains data on: the time the web scraper refreshed the page and collected data and the number of customers out at said time

Methods:

The following methods turn raw county-level data into approximations of aggregated outage-level data. **Methods are applied mutually exclusively to each county meaning outages are assumed to not cross county borders.**

Do's:

- Source: "[Spatiotemporal distribution of power outages with climate events and social vulnerability in the USA](#)" (Vivian Do, et al)
- "A power outage event occurs whenever the percent of customers without power met or exceeded 0.1% of the county customers" (Do, et al)
- We interpret this as an outage event starting if and only when over 0.1% of a county's customers are without power. The outage event is sustained until the percentage of customers without power falls below 0.1%.
- Example:

- The following a snapshot of raw county-level data for Flint EMC's Taylor county.
 - From the right, the columns are: Timestamp, customersServed, and customersOutNow.

372	Taylor	1	1	2539	07-02-2023 14:25:23
390	Taylor	7	7	2539	07-02-2023 23:46:45
392	Taylor	7	7	2539	07-02-2023 23:53:57
396	Taylor	7	7	2539	07-03-2023 05:23:35
401	Taylor	7	7	2539	07-03-2023 05:31:38
405	Taylor	7	7	2539	07-03-2023 06:01:51
409	Taylor	7	7	2539	07-03-2023 06:08:46
413	Taylor	7	7	2539	07-03-2023 06:16:40
417	Taylor	9	9	2539	07-03-2023 06:46:45
421	Taylor	9	9	2539	07-03-2023 06:55:40
425	Taylor	9	9	2539	07-03-2023 07:10:45
428	Taylor	3	3	2539	07-03-2023 07:25:41
431	Taylor	3	3	2539	07-03-2023 07:40:38
434	Taylor	1	1	2539	07-03-2023 07:55:39

- Since 0.1% of 2539 customers is 2.5 people, if the number of customers out exceeds 2.5, Do's method considers it the start of an outage. As highlighted, an outage starts at 7-02-2023 23:46:45 and persists until 7-03-2023 07:55:39 where the number of customers out falls below 2.5.
 - We record an outage occurred in Taylor county for 8 hours and 9 minutes

- Flaws: This method assumes that only one outage can occur in a county at a time.

Ganz:

- Source: "[Socioeconomic vulnerability and differential impact of severe weather-induced power outages](#)" (Scott Ganz, et al)
- "Outage duration for each affected country is computed as the duration of time that elapses between the time at which the number of disrupted customers reaches its peak level and the time it drops below 5% of the customers served." (Ganz, et al)
- We interpret this as a set of contiguous time snapshot rows being part of an outage if there is a local max of customers out above 5% of the customers served that falls to below 5% of the customers served.
- Example:
 - This is the a snapshot of a different value part of the Flint EMC dataset (data rows overflowed, so screenshot is split into two halves).

customersOutNow	customersServed	timestamp			
609	2565	09-27-2024 12:07:30			
612	2565	09-27-2024 12:22:41			
623	2564	09-27-2024 12:37:39	432	2564	09-27-2024 17:52:30
601	2564	09-27-2024 12:52:38	430	2564	09-27-2024 18:20:29
595	2564	09-27-2024 13:07:41	430	2564	09-27-2024 18:22:23
595	2564	09-27-2024 13:22:35	430	2564	09-27-2024 18:38:06
579	2564	09-27-2024 13:37:42	430	2564	09-27-2024 18:52:35
581	2564	09-27-2024 13:52:41	430	2564	09-27-2024 19:07:28
566	2564	09-27-2024 14:07:35	430	2564	09-27-2024 19:22:39
566	2564	09-27-2024 14:22:41	430	2564	09-27-2024 19:37:25
566	2564	09-27-2024 14:37:20	430	2564	09-27-2024 19:52:38
566	2564	09-27-2024 14:52:18	430	2564	09-27-2024 19:52:38
565	2564	09-27-2024 15:07:25	391	2564	09-27-2024 20:07:31
472	2564	09-27-2024 15:22:24	393	2564	09-27-2024 20:22:30
464	2564	09-27-2024 15:37:27	393	2564	09-27-2024 20:37:26
464	2564	09-27-2024 15:52:29	393	2564	09-27-2024 20:52:33
464	2564	09-27-2024 16:07:25	393	2564	09-27-2024 21:07:32
464	2564	09-27-2024 16:22:21	393	2564	09-27-2024 21:07:32
464	2564	09-27-2024 16:37:23	1	2564	09-27-2024 21:22:34
464	2564	09-27-2024 16:52:21	1	2564	09-27-2024 21:37:26
464	2564	09-27-2024 17:07:34			
452	2564	09-27-2024 17:22:23			
453	2564	09-27-2024 17:37:27			

- 5% of ~2550 = 128.5 customers
- The local max above the 5% threshold is 623 customers starting at 09-27-2024 12:37:39.
- The number of customers out continues until 09-27-2024 21:22:34 where it falls way below the 5% threshold.
- The outage is then calculated to start at 09-27-2024 12:37:39 and ends at 09-27-2024 21:22:34 making the duration 8 hours and 45 minutes long with the number of people affected to be within the hundreds.
- Flaws: Like Do's, Ganz method assumes that only one outage occurs at a time.

Mixed Threshold:

- Source: GROWER original method
- Percentage threshold:
 - Assuming that an outage can have x percent of fluctuation of customers affected between two consecutive timestamps
- Absolute threshold
 - Assuming that an outage can have an absolute difference up to x customers affected between two consecutive timestamps

- Within two consecutive timestamps where the number of affected customers **increase**, if the increase exceeds both the percentage threshold and absolute threshold, we consider a new outage has started while the previous one is ongoing.
- Likewise, within two consecutive timestamps where the number of affected customers **decreases**, if the decrease exceeds both the percentage threshold and the absolute threshold, we consider an outage to have ended while the previous ones are ongoing.
- Example:
 - From AEP Texas

utility	state	customers_affected	timestamp
AEP Texas	Texas	20	0:00
AEP Texas	Texas	100	0:15
AEP Texas	Texas	90	0:30
AEP Texas	Texas	80	0:45
AEP Texas	Texas	50	1:00
AEP Texas	Texas	40	1:15
AEP Texas	Texas	30	1:30
AEP Texas	Texas	0	1:45

- Percentage threshold = 10% and the absolute threshold = 10.
 - The number of customers out starts at 20 at timestamp 0:00 which exceeds both the percentage threshold and absolute threshold. Thus, an outage with 20 people out (red) is marked.
 - At timestamp 0:15, the number of customers out goes up 80 which exceeds both the 10% threshold ($20 * .1 = 2$) and the absolute threshold of 10 people. Thus, we indicate a new outage with 80 people out has begun at 0:15 (blue).
 - For timestamp 0:30 and 0:45, the decrease in customers out is 10 which does not exceed both the percentage threshold and the absolute threshold (not greater than the absolute threshold of 10).
 - Thus, the blue outage is marked as continuing with decreasing customers.
 - At timestamp 1:00, the number of customers falls by 30 which exceeds both thresholds of the blue outage.
 - This means an outage instance has changed status.
 - While the decrease of 30 isn't enough to end the blue outage, it is enough to end the red outage.
 - Thus, the red outage ends at 1:00 while 10 leftover customers are removed from the blue outage

- From timestamp 1:15 and onward, the decrease in customers out does not exceed both thresholds until 1:45 where it hits zero meaning the blue outage persevered until 1:45.
- Pros: Mixed threshold is able to record concurrent outages

PELT Changepoints Detection:

- Source:
<https://www.sciencedirect.com/science/article/pii/S0165168419303494?via%3Dihub>
- **Segmentation:**
 - The algorithm divides the time series into segments where the properties (mean, variance, etc.) are constant within each segment but change at the boundaries (change points)
- **Cost Function:**
 - The cost function, $C(t)$, is defined for each segment (t) and is typically based on the chosen model
 - Example: Sum of squared deviations for L2 norm

$$C(t) = \sum_{i=t_1}^{t_2} (y_i - \hat{y})^2$$

- For a segment $([t_1 - t_2])$ in the L2 norm model, where (y_i) is the data point and $\hat{y}$ is the mean of the segment
- **Penalty Term:**
 - The total cost includes a penalty term beta for each change point to control the complexity of the model.

$$\text{Total Cost} = \sum_{\text{segments}} C(t) + \beta \times (\text{number of change points})$$

- The penalty term helps balance between fitting the data well and avoiding too many change points (overfitting).
- **Cost Functions Used:**
 - **L1** - Based on the L1 norm (absolute distance). Example: The distance between 1 and 2 is $|1 - 2| = 1$.
 - **L2** - Based on the L2 norm (Euclidean distance). Example: The distance between 1 and 2 is $\sqrt{(1 - 2)^2} = 1$.
 - **rbf** - Utilizes Radial Basis Functions (RBF) to capture non-linear changes. Utilizes the distance from a central point and can model data. Example: $[1, 1.5, 2, 2.5, 5, 5.5, 6, 6.5] \gg [1, 1.5, 2, 2.5], [5, 5.5, 6, 6.5] \gg$ distance from central point of 1.75 and 5.75 for each segment.

- **normal** - Assumes the data follows a normal distribution, detecting changes in both mean and variance. Example: Useful when data has varying averages and spreads.
- **rank** - A non-parametric method that detects changes based on ranks. Example: raw data - [4, 3, 2, 1, 10, 9, 8, 7] >> rank data - [4, 3, 2, 1, 8, 7, 6, 5] >> change point at position 4
- Example:
 - Consider a time series: [1, 1, 2, 2, 10, 10, 50, 50, 2, 2, 1, 1]
 - High Penalty:
 - Detects fewer change points:
[1, 1, 2, 2, **10**, 10, **50**, 50, 2, 2, 1, 1]
Change points detected at 10 and 50.
 - Low Penalty:
 - Detects more change points:
[1, 1, **2**, 2, **10**, 10, **50**, 50, **2**, 2, **1**, 1]
Change points detected at 2, 10, 50, 2, and 1.